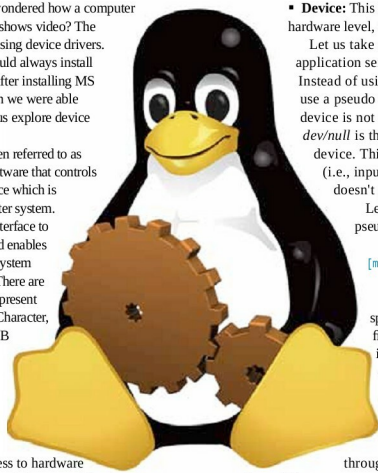
An Introduction to Device Drivers in the Linux Kernel

In the article 'An Introduction to the Linux Kernel' in the August 2014 issue of OSFY, we wrote and compiled a kernel module. In the second article in this series, we move on to device drivers.

Have you ever wondered how a computer plays audio or shows video? The answer is: by using device drivers. A few years ago we would always install audio or video drivers after installing MS Windows XP. Only then we were able to listen the audio. Let us explore device drivers in this column.

A device driver (often referred to as 'driver') is a piece of software that controls a particular type of device which is connected to the computer system. It provides a software interface to the hardware device, and enables access to the operating system and other applications. There are various types of drivers present in GNU/Linux such as Character, Block, Network and USB drivers. In this column, we will explore only character drivers.



Character drivers are the most common drivers. They provide unbuffered, direct access to hardware devices. One can think of character drivers as a long sequence of bytes -- same as regular files but can be accessed only in sequential order. Character drivers support at least the *open()*, *close()*, *read()* and *write()* operations. The text console, i.e., */dev/console*, serial consoles */dev/stty**, and audio/video drivers fall under this category.

To make a device usable there must be a driver present for it. So let us understand how an application accesses data from a device with the help of a driver. We will discuss the following four major entities.

- **User-space application:** This can be any simple utility like *echo*, or any complex application.
- **Device file:** This is a special file that provides an interface for the driver. It is present in the file system as an ordinary file. The application can perform all supported operation on it, just like for an ordinary file. It can *move*, *copy*, *delete*, *rename*, *read* and *write* these device files.
- **Device driver:** This is the software interface for the device and resides in the kernel space.
- **Device:** This can be the actual device present at the hardware level, or a pseudo device.

Let us take an example where a user-space application sends data to a character device. Instead of using an actual device we are going to use a pseudo device. As the name suggests, this device is not a physical device. In GNU/Linux */dev/null* is the most commonly used pseudo device. This device accepts any kind of data (i.e., input) and simply discards it. And it doesn't produce any output.

Let us send some data to the */dev/null* pseudo device:

```
[mickey]$ echo -n 'a' > /dev/null
```

In the above example, *echo* is a user-space application and *null* is a special file present in the */dev* directory. There is a *null* driver present in the kernel to control the pseudo device.

To send or receive data to and from the device or application, use the corresponding device file that is connected to the driver through the Virtual File System (VFS) layer. Whenever an application wants to perform any operation on the actual device, it performs this on the device file. The VFS layer redirects those operations to the appropriate functions that are implemented inside the driver. This means that whenever an application performs the *open()* operation on a device file, in reality the *open()* function from the driver is invoked, and the same concept applies to the other functions. The implementation of these operations is device-specific.

Major and minor numbers

We have seen that the *echo* command directly sends data to the device file. Hence, it is clear that to send or receive data to and from the device, the application uses special device files. But how does communication between the device file and the driver take place? It happens via a pair of numbers referred to as 'major' and 'minor' numbers.

The command below lists the major and minor numbers associated with a character device file: